

Deep Learning

5. Images et réseaux de neurones convolutifs

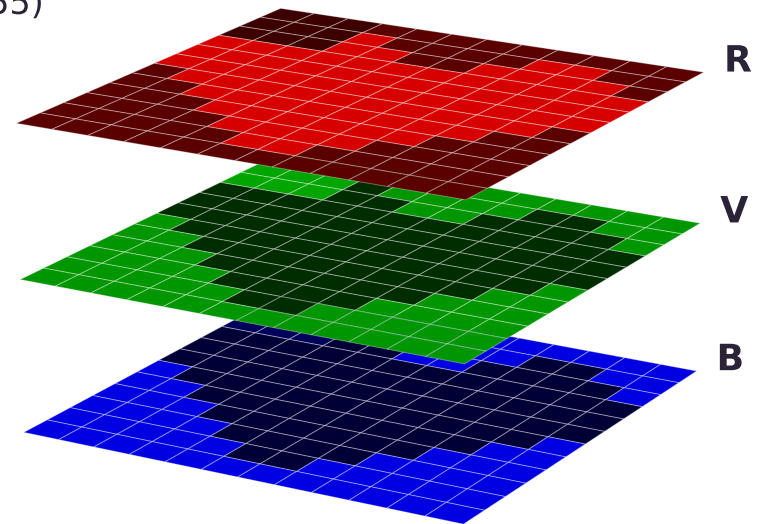
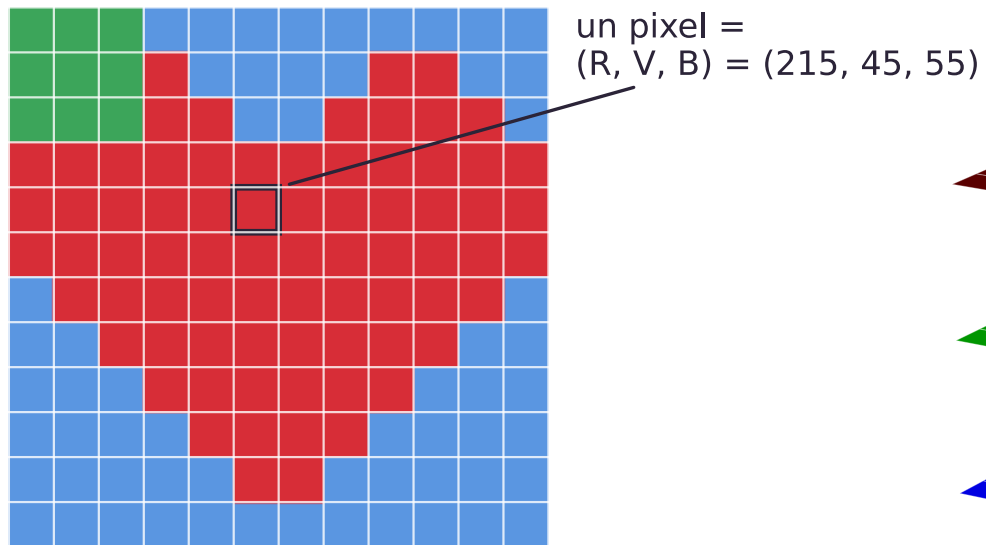
Romain Tavenard

NB : la plupart des figures de ces diapositives proviennent de *Dumoulin & Visin. "A guide to convolution arithmetic for deep learning". 2016*

Préambule : qu'est-ce qu'une image ?

Une image = une grille de pixels

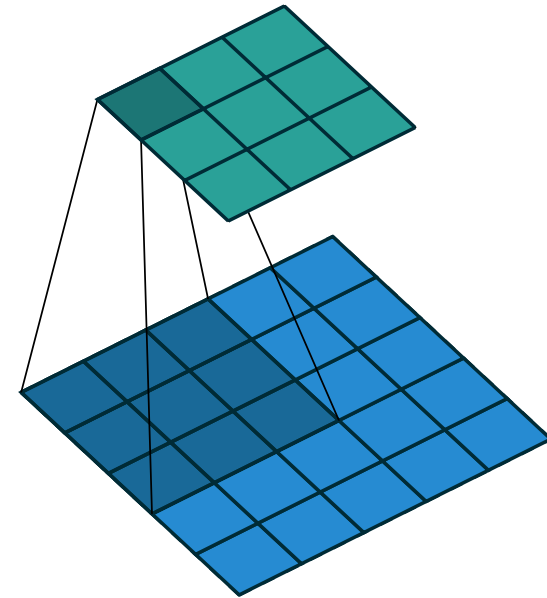
Un pixel = 3 valeurs de luminance (R, V, B)



hauteur $\times$ largeur $\times$ 3 canaux

L'opérateur de convolution

- Convolution 2D
 - Bleu : image d'entrée
 - Gris : noyau de convolution
 - Cyan : carte d'activation
- Opération de convolution = produit scalaire entre
 - le noyau de convolution (aussi appelé filtre)
 - une sous-partie de l'entrée



3	3	2	1	0
0	0 ₀	1 ₁	3 ₂	1
3	1 ₂	2 ₂	2 ₀	3
2	0 ₀	0 ₁	2 ₂	2
2	0	0	0	1

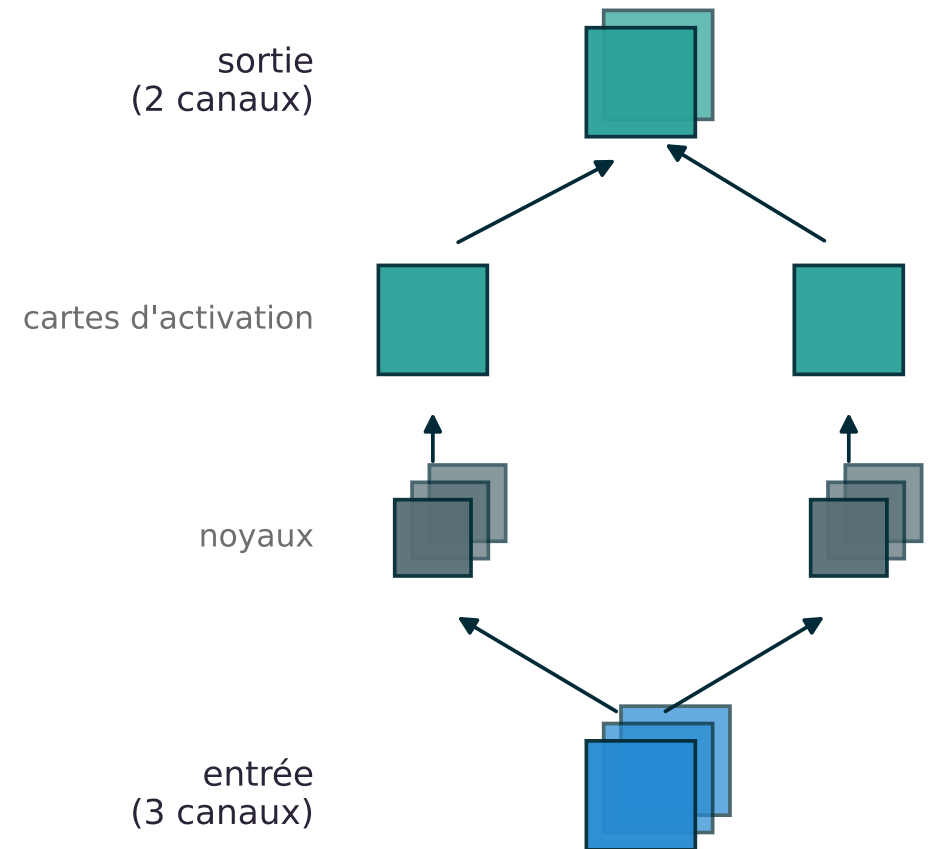
12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

Couches convolutives dans un réseau de neurones (1/2)

- Une couche convolutive est composée de
 - noyaux de convolution
 - biais (1 par noyau)
 - une fonction d'activation
- Utile car
 - réduit le nombre de paramètres (**partage de paramètres** : le même noyau est réutilisé partout)
 - *par ex.* neurone dense sur 224×224 : ≈ 50000 poids
 - contre 3×3 pour un noyau : 9
 - encode l'**équivariance par translation** (une translation de l'entrée induit une translation de la sortie)

Couches convolutives dans un réseau de neurones (2/2)

- Cas de plusieurs canaux d'entrée
 - on somme la réponse sur tous les canaux
- Cas de plusieurs noyaux
 - chaque noyau produit un canal de sortie



Couches convolutives : hyperparamètres

- Padding*

0	0	0	0	0	0	0
0	3	3	2	1	0	0
0	0	0 ₀	1 ₁	3 ₂	1	0
0	3	1 ₂	2 ₂	2 ₀	3	0
0	2	0 ₀	0 ₁	2 ₂	2	0
0	2	0	0	0	1	0
0	0	0	0	0	0	0

6.0	14.0	17.0	11.0	3.0
14.0	12.0	12.0	17.0	11.0
8.0	10.0	17.0	19.0	13.0
11.0	9.0	6.0	14.0	12.0
6.0	4.0	4.0	6.0	4.0

padding="same"

3	3	2	1	0
0	0 ₀	1 ₁	3 ₂	1
3	1 ₂	2 ₂	2 ₀	3
2	0 ₀	0 ₁	2 ₂	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

padding="valid"

- Strides*

3	3	2	1	0
0 ₀	0 ₁	1 ₂	3	1
3 ₂	1 ₂	2 ₀	2	3
2 ₀	0 ₁	0 ₂	2	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

stride=1

3	3	2 ₀	1 ₁	0 ₂
0	0	1 ₂	3 ₂	1 ₀
3	1	2 ₀	2 ₁	3 ₂
2	0	0	2	2
2	0	0	0	1

12.0	17.0
9.0	14.0

stride=2

Calculer la taille d'une carte d'activation

- Hypothèse : pas de *padding* (“valid”), *strides* unitaires

$$W_{\text{out}} = W_{\text{in}} - W_k + 1$$

$$H_{\text{out}} = H_{\text{in}} - H_k + 1$$

3	3	2	1	0
0	0 ₀	1 ₁	3 ₂	1
3	1 ₂	2 ₂	2 ₀	3
2	0 ₀	0 ₁	2 ₂	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

La sortie rétrécit de (taille du noyau – 1)

Exemple :

- Entrée : 28×28 (chiffre MNIST)
- Noyau : 5×5
- Sortie : $(28 - 5 + 1) \times (28 - 5 + 1) = 24 \times 24$

Avec padding=“same” : la sortie reste 28×28

Couches de *pooling* (ou sous-échantillonnage)

- Pourquoi faire du *pooling* ?
 - réduit la taille spatiale → moins de calculs
 - robustesse aux petits décalages (conv = équivariante, pooling → invariant)
- *Max pooling / Average pooling*
- Hyperparamètre
 - taille du pooling

3	3	2	1	0	0
0	1	3	1	3	1
2	2	3	2	0	0
2	2	2	0	0	0
1	0	1	3	2	2
3	2	0	3	0	1

3.0	3.0	3.0
2.0	3.0	0.0
3.0	3.0	2.0

Exemple de max pooling

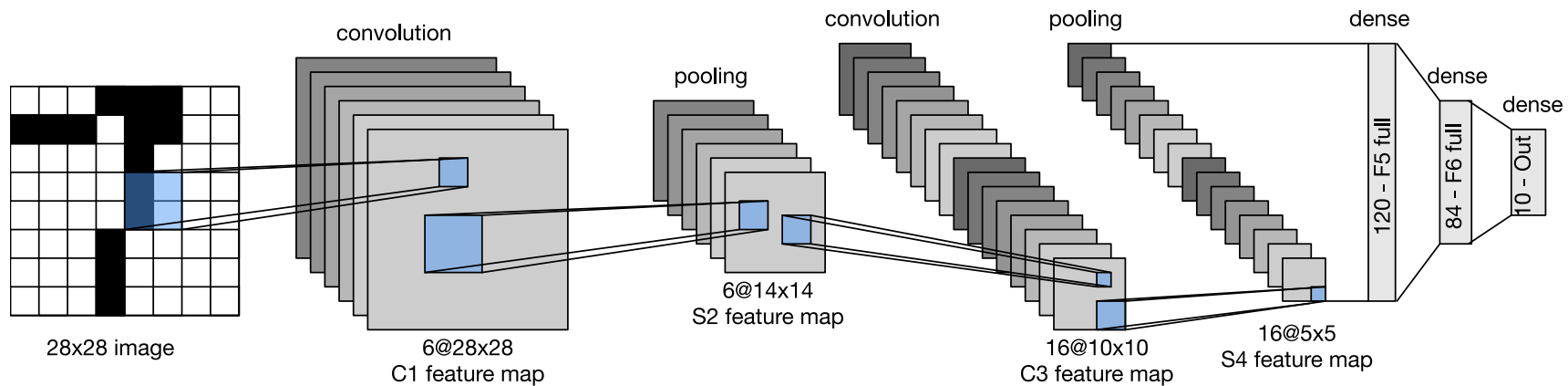
LeNet (1989)



Jeu de données MNIST

10 classes

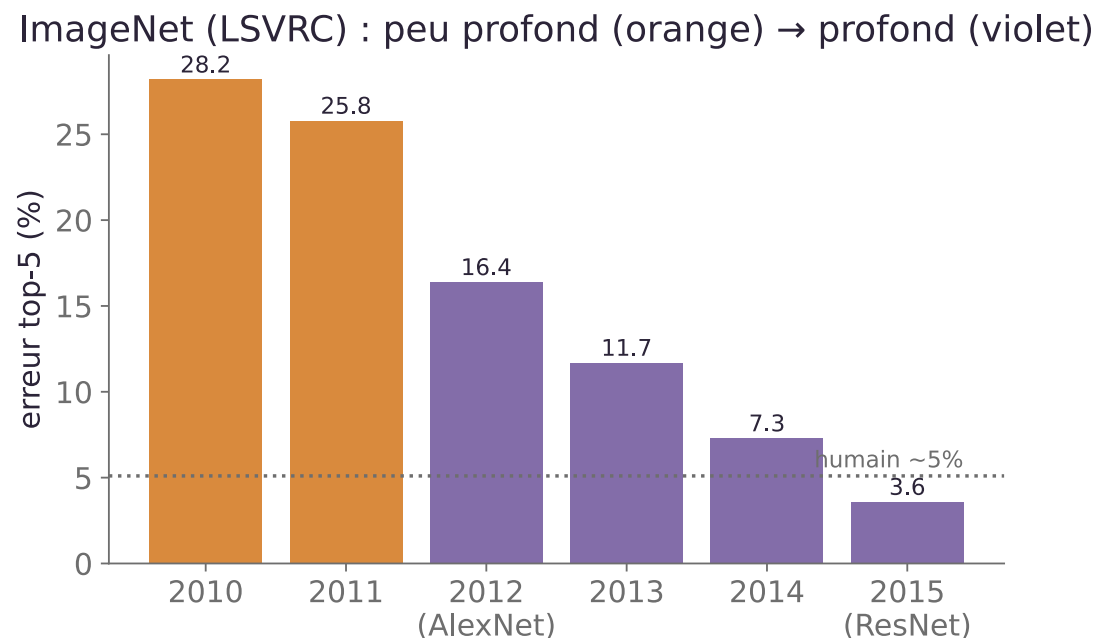
60 000 images



Architecture LeNet-5 (LeCun et al., 1989). Source : Wikipedia

2012–2015 : une amélioration radicale des performances

- ImageNet : 15M images, 22k classes
 - LSVRC : sous-ensemble d'ImageNet (1,2M images, 1k classes)
 - Ingrédients clés :
 - des réseaux plus profonds (⚠ disparition des gradients)
 - régularisation (BatchNorm, augmentation de données...)

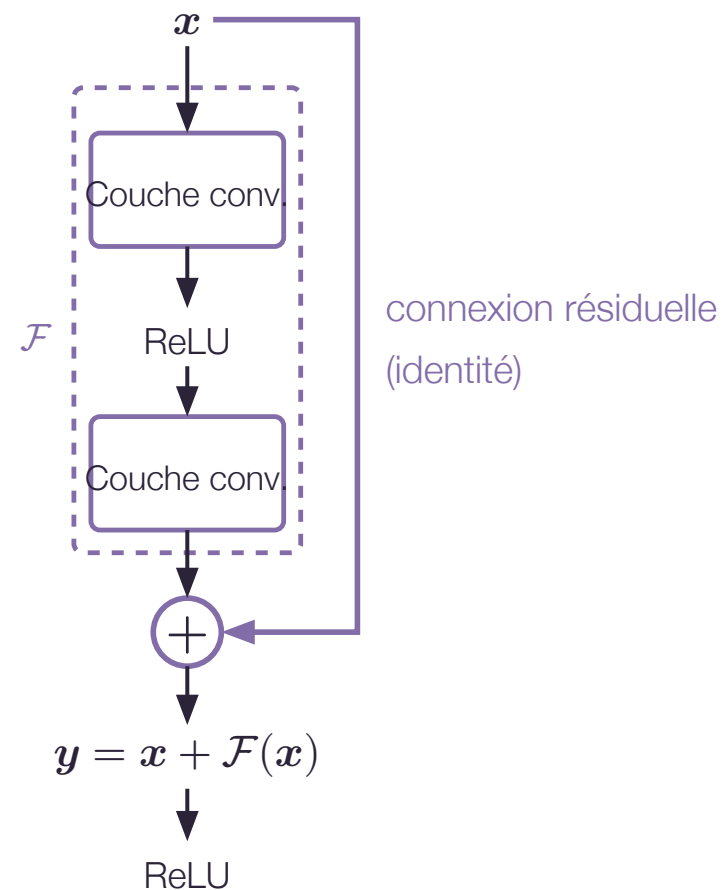


ResNet (2015) — vraiment très profond

- **Problème** : au-delà de 20 couches, plus profond = **pire** (disparition des gradients)
- **Connexion résiduelle** :

$$y = x + \mathcal{F}(x)$$

- rien d'utile à ajouter ? on sort $x \rightarrow$ la profondeur ne nuit jamais
- a permis des réseaux de **100+ couches**



Bloc résiduel : la connexion identité laisse les gradients contourner les couches de poids

ResNet (2015) — vraiment très profond

Pourquoi la connexion résiduelle corrige la disparition des gradients

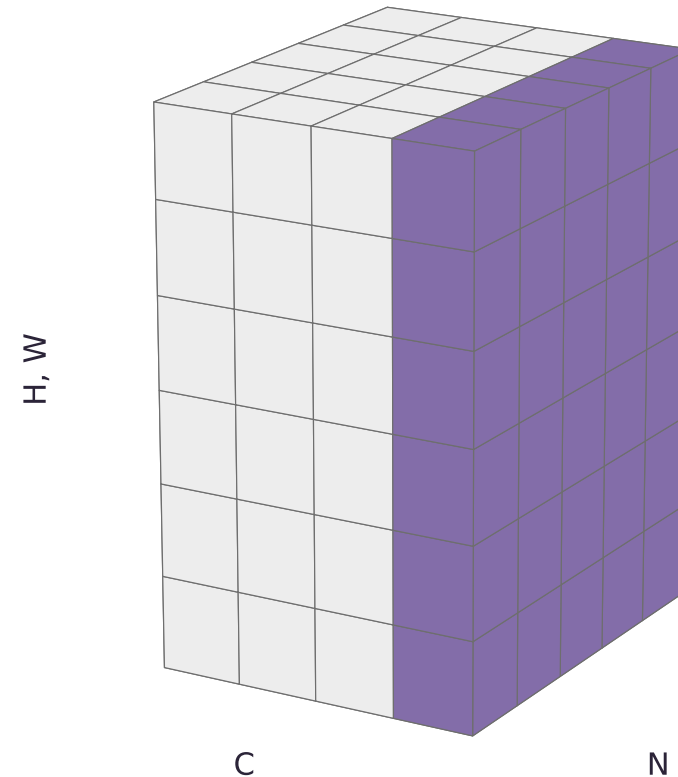
- Le gradient qui remonte à travers un bloc résiduel s'écrit :

$$\frac{\partial y}{\partial x} = \underbrace{1}_{\text{raccourci identité}} + \frac{\partial \mathcal{F}(x)}{\partial x}$$

- le **“1” est essentiel** : même si $\partial \mathcal{F} / \partial x$ est minuscule, le gradient reste ≈ 1 (et non ≈ 0)
- empiler L blocs $\rightarrow$ produit de termes $(1 + \dots) \rightarrow$ **pas d'effondrement vers zéro**

Batch Normalization pour les images

- Rappel : BatchNorm
normalise les activations
(moyenne nulle, variance
unitaire), puis les remet à
l'échelle avec γ, β appris
- Pour un batch d'images
(tenseur $N \times C \times H \times W$)
 - une paire moyenne/
variance **par canal** C
- Effet : stabilise/accélère
l'entraînement ; agit aussi
comme un régularisateur
(léger)



*Batch Norm : pour chaque canal
(tranche violette), les statistiques sont
calculées sur les dimensions batch et
spatiales (N, H, W)*

Régularisation : augmentation de données

- Rappel : les grands réseaux sur-apprennent → il faut régulariser (early stopping, Dropout, BatchNorm)
- Images : une astuce supplémentaire, très efficace :
l'**augmentation de données**
- Principe : générer des exemples d'entraînement virtuels

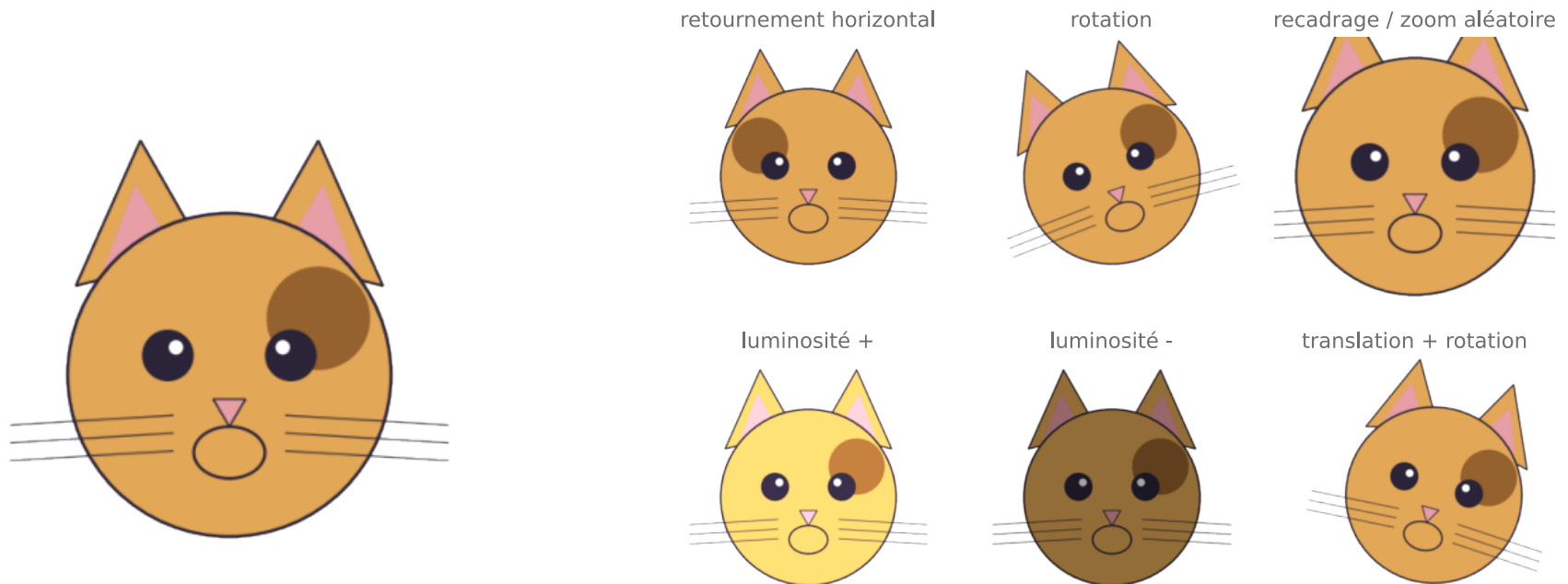


Image originale (étiquette : “chat”)

6 versions augmentées (même étiquette)

Conclusion

- Les couches convolutives sont les briques de base des CNN
- Les couches de pooling réduisent la taille spatiale et apportent une robustesse aux petits décalages
- BatchNorm stabilise l'entraînement et agit comme un régularisateur léger
- L'augmentation de données est une technique de régularisation très efficace pour les images